

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 5

Реализация межсервисного взаимодействия посредством очередей сообщений

Выполнила:

Сачук А. А.

Группа БР1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Необходимо подключить и настроить брокер сообщений RabbitMQ или Kafka, а также реализовать межсервисное взаимодействие через очередь сообщений. В работе был выбран RabbitMQ, так как его достаточно удобно использовать для обмена событиями между сервисами и проверки результата через логи приложения.

Ход работы

В качестве приложения использовался backend для сервиса рецептов. До выполнения задания сервисы взаимодействовали в основном через обычные HTTP-запросы. Для Д35 была добавлена асинхронная передача событий через RabbitMQ.

RabbitMQ был добавлен в docker-compose.yml. Для обмена сообщениями была создана очередь recipe-events. Через эту очередь один сервис может отправлять событие, а другой сервис получать его и выполнять нужное действие без прямой зависимости между сервисами.

Реализованное взаимодействие

В работе были выделены два основных участника обмена сообщениями:

- social-service - отправляет события при лайках, сохранениях и комментариях;
- recipe-service - получает события из очереди и обновляет счетчики рецептов, например likes_count и comments_count.

Таким образом, действия пользователя в social-service приводят к публикации события в RabbitMQ. После этого recipe-service получает событие из очереди recipe-events и обновляет данные по рецепту у себя. Такой подход уменьшает связанность сервисов: social-service не обязан напрямую обращаться к recipe-service для каждого действия.

Основные события

Для обмена были описаны события, связанные с действиями пользователя: создание лайка, сохранение рецепта и создание комментария. Формат события содержит тип события, идентификатор рецепта и дополнительные данные, которые нужны получателю для обработки.

Событие	Отправитель	Получатель	Назначение
recipe.liked	social-service	recipe-service	Обновление количества лайков
recipe.saved	social-service	recipe-service	Фиксация сохранения рецепта
comment.created	social-service	recipe-service	Обновление количества комментариев

Проверка работы

Работа API проверялась через Postman. Были выполнены запросы регистрации, авторизации, создания рецепта, получения списка рецептов, фильтрации, получения деталей рецепта, создания комментария и сохранения рецепта. По результатам запуска коллекции все тесты прошли успешно, ошибок не возникло.

После настройки очереди межсервисное взаимодействие стало работать асинхронно. Это значит, что сервис-отправитель публикует сообщение и продолжает работу, а сервис-получатель обрабатывает событие отдельно. Такой вариант удобен для действий, которые не требуют немедленного ответа пользователю, например обновления счетчиков.

Вывод

В ходе выполнения домашнего задания был подключен RabbitMQ и реализовано межсервисное взаимодействие через очередь сообщений recipe-events. Social-service публикует события, связанные с действиями пользователя, а recipe-service получает эти события и обновляет данные рецептов. В

результате сервисы стали меньше зависеть друг от друга, а часть логики была вынесена в асинхронную обработку. Также работа API была проверена в Postman, тестовые запросы выполнились успешно.